

灵活用工结算平台完整研发方案

文档版本：V4.0

编制日期：2026年05月15日

项目代号：FLEX-SETTLE

密级：客户对外

目录

- 1. [项目概述](#1-项目概述)
- 2. [业务需求与功能范围](#2-业务需求与功能范围)
- 3. [技术架构设计](#3-技术架构设计)
- 4. [数据库设计](#4-数据库设计)
- 5. [核心业务流程](#5-核心业务流程)
- 6. [开发计划与里程碑](#6-开发计划与里程碑)
- 7. **报价与付款方案（首期40%）**
- 8. [部署与运维方案（Docker）](#8-部署与运维方案docker)
- 9. [质量保证与风险应对](#9-质量保证与风险应对)
- 10. [交付物清单](#10-交付物清单)

1. 项目概述

1.1 建设目标

建设一套面向灵活用工场景的**结算管理平台**，帮助企业完成对自由职业者（骑手、兼职等）的任务发布、接单执行、佣金结算、个税代扣、发票获取、资金对账等全流程线上化管理。平台采用多租户架构，支持多家企业独立入驻、数据物理隔离。

1.2 核心价值

- 合规高效**：自动完成个税计算与申报，打通银行虚拟账户实现秒级代付，形成“业务+资金+发票”三流合一。
- 成本可控**：通过标准化 SaaS 模式降低企业自建成本，30 万元内完成 4 个月定制开发。
- 易于部署**：采用 Docker 容器化，单机即可运行，无需 Kubernetes 复杂运维。

1.3 非功能性目标

2. 业务需求与功能范围

2.1 角色与用例

2.2 功能模块清单（不含电子签名）

3. 技术架构设计

3.1 技术栈总览

3.2 多租户设计

- **隔离策略**：Schema-per-tenant。每个租户独立 Schema `tenant_{code}`，公共表在 `public` Schema。
- **租户识别**：前端请求头 `X-Tenant-ID` → 后端中间件校验 → 存入 `context` → GORM 回调动态设置 `search_path`。
- **连接池**：每个租户使用独立连接池（`gorm-multitenancy` 管理）。

3.3 后端模块结构（单体+内部模块化）

```
backend/
├─ cmd/server/main.go          # 入口（同时启动 API 与 Worker）
├─ internal/
│  ├─ handler/                 # HTTP 控制器
│  ├─ service/                 # 业务逻辑
│  ├─ repository/              # 数据访问
│  ├─ model/                   # GORM 模型
│  ├─ middleware/              # 租户/认证/日志/限流
│  ├─ worker/                  # 后台任务（消费 RabbitMQ）
│  └─ scheduler/               # 定时任务（如每日对账）
├─ pkg/                        # 公共库 (errno, utils)
├─ migrations/                 # 迁移脚本 (public + tenant 模板)
└─ configs/config.yaml
```

3.4 前端结构（Feature-Sliced）

```
frontend/
├─ src/
│  ├─ app/                     # 路由、全局 store、入口
│  ├─ pages/                   # 页面组件（按模块）
│  ├─ features/                 # 独立功能（上传、搜索）
│  ├─ entities/                # 实体类型定义
│  └─ shared/                   # UI 组件、API 客户端、工具
├─ Dockerfile
└─ nginx.conf
```

3.5 API 设计规范

- RESTful 风格，统一响应格式：

```
{
  "code": 0,
  "message": "success",
  "data": {},
  "trace_id": "xxx"
}
```

- 错误码： 1xxx 参数错误， 2xxx 认证/权限， 3xxx 业务异常， 5xxx 系统错误。
- 认证：JWT (Access Token 2h, Refresh Token 7d)，登录后返回。

4. 数据库设计

4.1 ERD 核心表（租户 Schema）

```
erDiagram
    tenants ||--o{ tasks : owns
    tenants ||--o{ workers : employs
    tasks ||--o{ task_assignments : "assigns"
    workers ||--o{ task_assignments : "performs"
    task_assignments ||--o{ settle_details : "settles"
    settle_details }o--|| settle_batches : belongs
    workers ||--o{ fund_accounts : has
    fund_accounts ||--o{ fund_transactions : produces
    tenants ||--o{ invoices : requests
```

4.2 关键表结构（简化）

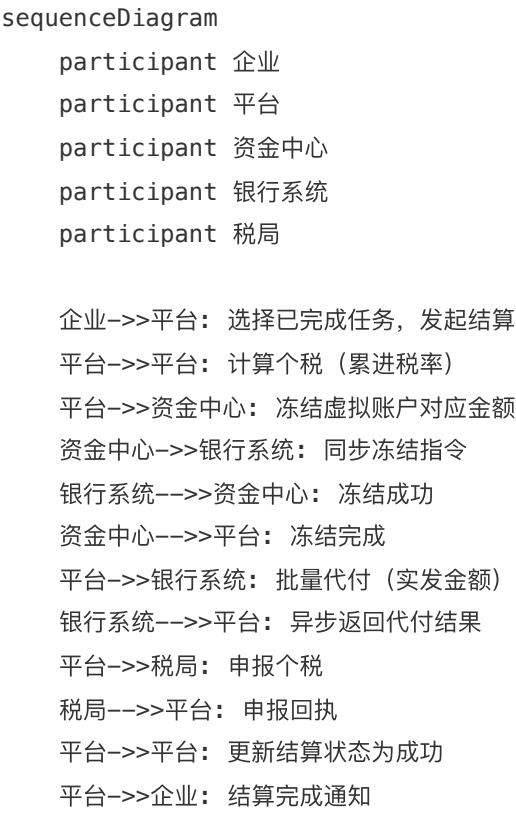
完整表结构及 SQL 脚本随交付物提供。

4.3 多租户迁移管理

- 使用 `golang-migrate` 管理 `public` 和租户模板。
- 新租户注册时自动执行：`CREATE SCHEMA tenant_xxx` → 运行模板迁移。

5. 核心业务流程

5.1 结算全流程



5.2 对账流程

- 每日凌晨 2:00 自动拉取银行对账单，与平台 fund_transactions 逐笔比对。
- 差异记录到 reconciliation_diff 表，并发送告警。

6. 开发计划与里程碑

总工期 **16 周**（4 个月），采用敏捷迭代。

6.1 阶段划分

6.2 时间节点示例

7. 报价与付款方案

7.1 总报价：30 万元（含税，包干价）

注：不含银行/税局等第三方接口的年度服务费，不含生产环境云资源费用。

7.2 付款计划（首期 40%）

7.3 包含与不包含

8. 部署与运维方案（Docker）

8.1 硬件推荐配置

- **最小**：4核 CPU，8GB 内存，50GB SSD（适合日均单量 < 1000）
- **推荐**：8核 CPU，16GB 内存，100GB SSD（日均单量 5000+）

8.2 部署架构（单机 Docker Compose）

```
# docker-compose.yml 核心服务
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: flex_settle
      POSTGRES_USER: flex
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - ./data/postgres:/var/lib/postgresql/data
  redis:
    image: redis:7-alpine
  rabbitmq:
    image: rabbitmq:3-management
  backend:
    build: ./backend
    depends_on: [postgres, redis, rabbitmq]
  worker:
    build: ./backend
    command: ["./worker"]
  frontend:
    build: ./frontend
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
```

8.3 数据备份策略

- PostgreSQL：每日凌晨 3:00 执行 `pg_dump` 并上传至 OSS，保留 30 天。
- Redis：开启 AOF + RDB。
- 容器日志：通过 `docker logs` 收集或映射到宿主机，使用 `logrotate` 轮转。

8.4 监控告警（轻量）

- 使用 `cAdvisor` + `Prometheus` + `Grafana` 可选（但不强制）。
- 必需的健康检查：每个容器提供 `/health` 端点，`docker-compose` 配置 `healthcheck` 。

9. 质量保证与风险应对

9.1 测试策略

9.2 风险应对表

10. 交付物清单

附录：联系与支持

- **技术支持：**项目交付后提供 1 个月免费远程缺陷修复，之后可按年收取 15% 维护费（可选）。
- **定制开发：**超出本方案范围的电子签名模块、K8s 迁移等，另行评估报价。